

3D-BBS: Global Localization for 3D Point Cloud Scan Matching Using Branch-and-Bound Algorithm

Koki Aoki¹, Kenji Koide², Shuji Oishi², Masashi Yokozuka², Atsuhiko Banno², and Junichi Meguro¹

Abstract—This paper presents an accurate and fast 3D global localization method, 3D-BBS, that extends the existing branch-and-bound (BnB)-based 2D scan matching (BBS) algorithm. To reduce memory consumption, we utilize a sparse hash table for storing hierarchical 3D voxel maps. To improve the processing cost of BBS in 3D space, we propose an efficient roto-translational space branching. Furthermore, we devise a batched BnB algorithm to fully leverage GPU parallel processing. Through experiments in simulated and real environments, we demonstrated that the 3D-BBS enabled accurate global localization with only a 3D LiDAR scan roughly aligned in the gravity direction and a 3D pre-built map. This method required only 878 msec on average to perform global localization and outperformed state-of-the-art global registration methods in terms of accuracy and processing speed.

I. INTRODUCTION

3D light detection and ranging sensor (LiDAR)-based localization has played an important role in the mobile robot. LiDAR-based global localization, which is the task of finding a robot's global pose in a pre-built map without any initial estimates, is effective in dense building environments where the global navigation satellite systems (GNSS) accuracy decreases due to the multipath and signal blockage. This technique is a key technology in robotics applications such as initial pose estimation. However, LiDAR-based global localization with a large map in real time is challenging because of the large search space.

There are several approaches to solving the global localization problem. Global registration methods [1], [2], [3] extract point features to describe local geometrical shapes and relationships. A transformation is then estimated by establishing point-to-point correspondences. Although these methods can robustly match a pair of 3D point clouds with outliers [1], they require considerable processing time for point correspondence estimation with a large map point cloud and often suffer from repeated environmental structures.

For scan-to-map matching, Hess et al. [4] proposed a 2D global localization method based on the branch-and-bound (BnB) algorithm. Their method used hierarchical occupancy grid maps to compute the upper bound of scan matching scores and efficiently prune unpromising candidate poses through hierarchical search space tree branching. This

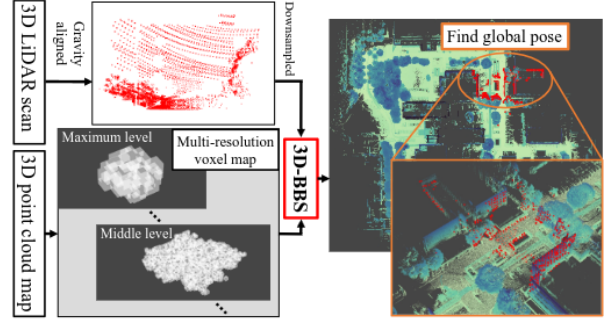


Fig. 1: Overview of the proposed global localization.

approach enables precise and fast global localization in 2D maps. However, these performances are difficult to achieve in 3D maps because memory consumption and processing costs drastically increase.

In this work, we propose a 3D global localization method using BnB-based scan matching (3D-BBS). Our method estimates the sensor pose on a 3D pre-built map with a single 3D LiDAR scan roughly aligned in the gravity direction. 3D-BBS reduces memory consumption and processing time using sparse map structures and GPU-accelerated batched BnB algorithm. In our experiments, 3D-BBS exhibits accurate and fast global localization in a second on average, and significantly outperforms the state-of-the-art global registration methods [1], [2], [3].

The main contributions of this work are as follows:

- We propose a batched BnB algorithm that quickly computes a large number of score calculations at once on the GPU.
- For a faster BnB algorithm, we employ a combination of the best-first search and roto-translational branching. We empirically confirmed that it can drastically reduce the processing time with a slight approximation.
- We utilize a 3D voxel map created with a sparse hash table that enables saving the memory required for a 3D multi-resolution voxel map. The code of the proposed algorithm is available on a GitHub repository¹.

II. RELATED WORK

A. Global Localization with Point Cloud Matching

For global pose estimation, many studies have utilized feature extraction of point clouds [5]. In this approach, descriptors that represent local geometrical shapes, such as

^{*}This work was supported in part by a project commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

¹Koki Aoki and Junichi Meguro are with the Department of Mechatronics Engineering, Meijo University, 1-501 Shiogamaguchi, Tempaku-ku, Nagoya 4688502, Japan, 223432001@ccmail.g.meijo-u.ac.jp

²Kenji Koide, Shuji Oishi, Masashi Yokozuka, and Atsuhiko Banno are with the Department of Information Technology and Human Factors, the National Institute of Advanced Industrial Science and Technology, Umezono 1-1-1, Tsukuba, 3050061, Ibaraki, Japan.

¹https://github.com/KOKIAOKI/3d_bbs

FPFH [6], and SHOT [7] are extracted to determine the point correspondences between point clouds. Subsequently, the relative pose between the point clouds is estimated using a robust pose estimator such as RANSAC [8]. TEASER [1] utilizes a graph-theoretic consensus finding algorithm that enables robust pose estimation for a large number of outliers. Although these methods achieve fast alignment in scan-to-scan global registration, the registration of scan to large map would require large processing time to find the correspondence.

Chen et al. [9] use the range images generated from LiDAR scans and a meshed map. Their method achieved global localization for various datasets using an observation model for a Monte Carlo localization (MCL) [10]. However, a large number of particles are required to initialize accurately, and convergence takes a long time.

Frame-based methods extract geometrical features to represent a whole point cloud frame and identify revisited places by finding frames with similar descriptors. Scan context [11] creates a database of descriptors based on the height of 3D LiDAR points and effectively detects the loops in a trajectory. OverlapNet [12] utilizes a deep neural network exploiting different cues generated from LiDAR data, and estimates the overlap rates and yaw angle between two point clouds. OverlapTransformer [13] uses only LiDAR data, and achieves loop closure candidates detection faster than OverlapNet. BoW3D [14] built a dictionary of bag-of-words based on Link3D [15] features representing LiDAR scan keypoints and achieved real-time performance in 6DoF loop closure. While frame-based methods require keyframes of LiDAR scans that cover the entire map, 3D-BBS directly uses 3D maps that have already been constructed.

B. Point Cloud Matching Using Branch-and-Bound Method

Several studies have used the BnB algorithm [16], [17] to quickly estimate the accurate global pose. Because the BnB algorithm is a full search algorithm, a pose with the global best matching cost can be estimated more reliably than with methods using Monte Carlo algorithms. Olsson et al. [18] propose a BnB-based registration algorithm that finds the global optima of the non-convex registration problems. Go-ICP [19] combines the iterative closest point (ICP) [20] with the BnB algorithm to find the globally optimal solution. However, these methods require an initial pose estimate. Hess et al. [4] develop BnB-based 2D scan matching by using hierarchical occupancy grid map sizes. Although this method enables accurate and fast global localization on 2D maps, the processing time increases drastically for 3D maps.

III. METHODOLOGY

A. Problem Setting

Inspired by [4], we propose a global localization framework, 3D-BBS, as shown in Fig. 1. The 3D global localization problem is defined as the task of finding the 6DoF sensor pose $\mathbf{x} = [x, y, z, \alpha, \beta, \gamma]^\top$, where (x, y, z) represents the translation, and (α, β, γ) denotes Euler angles corresponding to roll, pitch, and yaw, respectively. Let $\mathbf{T}_x$ be

the transformation matrix of $\mathbf{x}$. Given a 3D point cloud map $\mathcal{M} = \{\mathbf{m}_n \in \mathbb{R}^3 | n=1, \dots, N\}$ and 3D LiDAR scan points $\mathcal{S} = \{\mathbf{s}_k \in \mathbb{R}^3 | k=1, \dots, K\}$, 3D-BBS estimates the sensor pose $\mathbf{x}^*$ on the map. We assume that the gravity direction of $\mathcal{S}$ is roughly estimated, for example, by using an inertial measurement unit (IMU), and perform scan matching in full 4DoF (XYZ translation and yaw angle) with a small roll and pitch search range.

We review the outline of the 2D BBS technique in Sec. III-B, and then extend it to 3D in Sec. III-C to III-E.

B. 2D Branch-and-Bound Scan Matching

Hess et al. [4] proposed branch-and-bound scan matching (BBS) in the context of global localization for 2D SLAM loop detection. Let $M : \mathbb{R}^2 \rightarrow \mathbb{Z}$ be an occupancy grid map from a point on the grid into the occupancy value of the nearest grid with cell size r . Given 2D LiDAR scan points $\mathcal{S} = \{\mathbf{s}_k \in \mathbb{R}^2 | k=1, \dots, K\}$, the BBS estimates the global solution of the 3DoF sensor pose $\mathbf{x}^*$ as follows:

$$\mathbf{x}^* = \underset{\substack{x \in \{r i_x | i_x \in \mathbb{Z}, W_{\min}^x \leq r i_x \leq W_{\max}^x\} \\ y \in \{r i_y | i_y \in \mathbb{Z}, W_{\min}^y \leq r i_y \leq W_{\max}^y\} \\ \theta \in \{\delta(r) i_\theta | i_\theta \in \mathbb{Z}, W_{\min}^\theta \leq \delta(r) i_\theta \leq W_{\max}^\theta\}}}{\text{argmax}} \sum_{k=1}^K M(\mathbf{T}_x \mathbf{s}_k), \quad (1)$$

where K is the number of the input LiDAR scan, $\mathbf{T}_x$ is the 3 DoF transformation matrix of pose hypothesis $\mathbf{x} = [x, y, \theta]^\top$, and $(W_{\min}^x, W_{\max}^x)$, $(W_{\min}^y, W_{\max}^y)$, $(W_{\min}^\theta, W_{\max}^\theta)$ are translational and rotational search ranges, respectively.

Branch-and-bound algorithm: The BnB algorithm, which is a key component of BBS, uses a hierarchical tree search to quickly find a solution that is equivalent to the result of the full search. The tree structure and leaf nodes represent the search space and feasible solutions to the problem, respectively. Nodes in an upper level represent a set of child nodes (i.e., branching). At each non-leaf node, an upper bound estimate of the scores of its children is calculated. If the calculated upper bound is lower than the provisional best score, we can prune that node with children nodes without approximation (i.e., pruning). If a high provisional best score is found earlier, the BnB algorithm can prune many nodes, resulting in a large reduction of the computation cost. Therefore, the order of the search is important to maximize the performance of the BnB algorithm (i.e., search strategy).

Branch-and-bound scan matching [4]: To solve the problem of 2D global localization using the BnB algorithm, the pose space within the search range is represented as a tree structure. Let $l \in \mathbb{Z}, 0 \leq l \leq l_{\max}$ be the hierarchy level in the tree, where $l_{\max}$ is the maximum level. The tree is branched from large to small step sizes. The step size at each level is determined based on the cell size r of the occupancy grid map. 1) The step size is $r_l = 2^l r$. 2) The rotational step size $\delta(r)$ is common for each level and selected such that a point in the maximum scan range $d_{\max}$ does not move more than r as follows:

$$\delta(r) = \arccos\left(1 - \frac{r^2}{2d_{\max}^2}\right). \quad (2)$$

Each node of the tree is represented by $c = (c_x, c_y, c_\theta, c_l) \in \mathbb{Z}^4$, where c_x , c_y , and c_θ are the discrete parameters representing the sensor pose $\mathbf{x}_c$, and c_l is the level of the node c .

The tree is branched from the maximum level $l_{\max}$. An initial set $\mathcal{C}_{l_{\max}}$ is created as the direct product set of the translational and rotational initial sets:

$$\mathcal{C}_{l_{\max}} = \mathcal{C}_{l_{\max}}^x \times \mathcal{C}_{l_{\max}}^y \times \mathcal{C}_{l_{\max}}^\theta \times \{l_{\max}\}, \quad (3)$$

where the initial sets of each component are as follows:

$$\begin{aligned} w_{\min}^x &= \left\lfloor \frac{W_{\min}^x}{2^{l_{\max}} r} \right\rfloor, & w_{\max}^x &= \left\lceil \frac{W_{\max}^x}{2^{l_{\max}} r} \right\rceil, \\ w_{\min}^y &= \left\lfloor \frac{W_{\min}^y}{2^{l_{\max}} r} \right\rfloor, & w_{\max}^y &= \left\lceil \frac{W_{\max}^y}{2^{l_{\max}} r} \right\rceil, \\ w_{\min}^\theta &= \left\lfloor \frac{W_{\min}^\theta}{\delta(r)} \right\rfloor, & w_{\max}^\theta &= \left\lceil \frac{W_{\max}^\theta}{\delta(r)} \right\rceil, \\ \mathcal{C}_{l_{\max}}^x &= \{i_x \in \mathbb{Z} \mid w_{\min}^x \leq i_x \leq w_{\max}^x\}, \\ \mathcal{C}_{l_{\max}}^y &= \{i_y \in \mathbb{Z} \mid w_{\min}^y \leq i_y \leq w_{\max}^y\}, \\ \mathcal{C}_{l_{\max}}^\theta &= \{i_\theta \in \mathbb{Z} \mid w_{\min}^\theta \leq i_\theta \leq w_{\max}^\theta\}. \end{aligned} \quad (4)$$

A non-leaf node c branches into four children nodes as follows:

$$\mathcal{C}_{c_l} = \{(2c_x + j_x, 2c_y + j_y, c_\theta, c_l - 1) \mid (j_x, j_y) \in \{0, 1\}^2\}. \quad (6)$$

At each branched node, the scan matching score is calculated as follows:

$$score(c) = \sum_{k=1}^K M(\mathbf{T}_{\mathbf{x}_c} \mathbf{s}_k), \quad (7)$$

$$\mathbf{x}_c = [r_{c_l} c_x, r_{c_l} c_y, \delta(r) c_\theta]^\top. \quad (8)$$

To calculate the upper bound without effort to find the maximum score of the children, M_{precomp}^l with a cell size $2^l r$ is created at each level in advance. The upper bound can be efficiently computed as follows:

$$\overline{score}(c) = \sum_{k=1}^K M_{\text{precomp}}^{c_l}(\mathbf{T}_{\mathbf{x}_c} \mathbf{s}_k), \quad (9)$$

$$M_{\text{precomp}}^l(x, y) = \max_{\substack{x' \in [x, x+r(2^l-1)] \\ y' \in [y, y+r(2^l-1)]}} M(x', y'). \quad (10)$$

Algorithm 1 shows the outline of the 2D BBS algorithm. All nodes in $\mathcal{C}_{l_{\max}}$ are initially saved to queue C in the descending order of $score(c)$. To quickly select a provisional best score, the conventional method uses a depth-first search (DFS), which first selects nodes at a lower level. Moreover, $best_score$ is initialized by the score threshold $score_threshold$ to ignore nodes that have poor matches.

Extending this algorithm to 3D introduces two challenges. First, multi-resolution 3D voxel maps consume a significant amount of memory. Second, the number of candidate pose nodes grows exponentially as the dimension increases. To overcome these problems, we propose the 3D-BBS algorithm with sparse voxel maps, rotational branching, and batch processing.

Algorithm 1 BBS [4]

```

1:  $best\_score \leftarrow score\_threshold$ 
2: while  $C$  is not empty do
3:   Pop  $c$  from the queue  $C$ 
4:   if  $score(c) > best\_score$  then
5:     if  $c$  is a leaf node then
6:        $match \leftarrow c$ 
7:        $best\_score \leftarrow score(c)$ 
8:     else
9:       Branch: Split  $c$  into nodes  $\mathcal{C}_{c_l}$ 
10:      Compute and memorize a score for each element in  $\mathcal{C}_{c_l}$ 
11:      Push  $\mathcal{C}_{c_l}$  onto the queue  $C$ , sorted by depth level,
        maximum score last.
12: return  $best\_score$  and  $match$  when set.

```

C. Multi-resolution Voxel Map

To reduce the memory consumption for a 3D multi-resolution precomputed map, we introduce spatial hashing-based sparse voxel maps. The voxel resolution at each level is expressed as $r_l = 2^l r$, where r is the minimum voxel size. We create an array of hash buckets per level, and store 3D coordinate occupied voxels $\mathbf{v} = [v_x, v_y, v_z]^\top \in \mathbb{Z}^3$, $v_x = \lfloor x/r_l \rfloor$, $v_y = \lfloor y/r_l \rfloor$, $v_z = \lfloor z/r_l \rfloor$ to the hash bucket. The function $H^l : \mathbb{R}^3 \rightarrow \{0, 1\}$ is defined to search for the bucket storing $\mathbf{v}$ and return its binary occupancy. To search a hash bucket, we utilize a spatial hashing function $hash(\mathbf{v})$ that computes the index of the corresponding hash bucket as follows:

$$hash(\mathbf{v}) = f(\mathbf{v}) \bmod T_l, \quad (11)$$

where the function $f(\mathbf{v})$ computes a hash value from a tuple of integers [21], and T_l is the size of hash buckets at l . In hash buckets construction, we seek T_l until the hash collision rate becomes small (e.g. 0.1%). To avoid hash collisions, we implement the open addressing method; if a bucket is already occupied by another voxel, we seek an empty bucket while incrementing the bucket index.

For each resolution, the 3D voxel coordinates of all map points in $\mathcal{M}$ are calculated and stored in hash buckets. For upper bound calculation, the voxels $\mathcal{V}_{\text{add}}^{l,v} = \{[v_x - j_x, v_y - j_y, v_z - j_z]^\top \mid (j_x, j_y, j_z) \in \{0, 1\}^3\}$ adjacent to all occupied voxels are stored in hash buckets so that the upper bound estimate of a parent node exceeds the scores of its children.

Let $c = (c_x, c_y, c_z, c_\alpha, c_\beta, c_\gamma, c_l) \in \mathbb{Z}^7$ be a node with discrete parameters representing the 6DoF sensor pose $\mathbf{x}_c$. We compute the matching score of node c that simultaneously maintains the upper bound of scores in its children by using the precomputed map as follows:

$$score(c) = \overline{score}(c) = \sum_{k=1}^K H^{c_l}(\mathbf{T}_{\mathbf{x}_c} \mathbf{s}_k). \quad (12)$$

D. Branching Algorithm

Branching rule: In contrast to [8], which only performs the branching of translational components, we perform both translational and rotational branching. The rotational branching makes the rotational step size sparse and drastically reduces the number of initial nodes. Fig. 2 illustrates the proposed branching method. We have a translational search range equal to the bounding box of $\mathcal{M}$. In translational

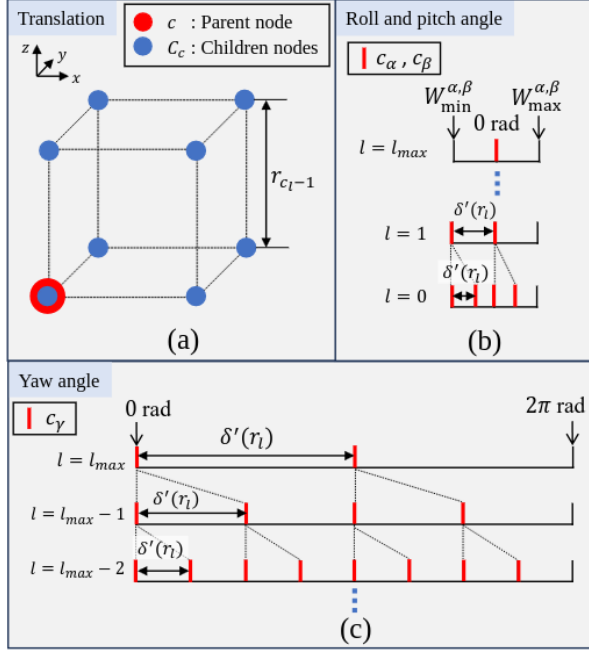


Fig. 2: Overview of the branching operation in the proposed method. (a) Translational branching. (b) Roll and pitch angle branching. (c) Yaw angle branching.

branching, a parent node branches into eight children nodes with step size r_{c_l-1} (Fig. 2 (a)). The roll and pitch components are searched within a small range ($W_{\min}^{\alpha, \beta}, W_{\max}^{\alpha, \beta}$) (Fig. 2 (b)). The yaw angle search range is ($W_{\min}^{\gamma}, W_{\max}^{\gamma}$) = ($0, 2\pi$) (Fig. 2 (c)). In rotational branching, we choose a step size for each level such that it becomes smaller than $\delta(r)$ and divide the search range into segments of the same size as follows:

$$\delta'(r_l) = \frac{W_{\max} - W_{\min}}{\left\lceil \frac{W_{\max} - W_{\min}}{\delta(r_l)} \right\rceil}. \quad (13)$$

The initial set $C_{l_{\max}}$ contains combinations of all poses within the search range at $l_{\max}$. $C_{l_{\max}}$ is initially saved to queue C in descending order of $score(c)$. Subsequently, a non-leaf node c popped from C is branched as follows:

$$C_{c_l} = \{(2c_x + j_x, 2c_y + j_y, 2c_z + j_z, a_\alpha c_\alpha + j_\alpha, a_\beta c_\beta + j_\beta, a_\gamma c_\gamma + j_\gamma, c_l - 1) | (j_x, j_y, j_z) \in \{0, 1\}, j_\alpha \in \{0, \dots, a_\alpha - 1\}, j_\beta \in \{0, \dots, a_\beta - 1\}, j_\gamma \in \{0, \dots, a_\gamma - 1\}\}, \quad (14)$$

where a_α, a_β and a_γ are the number of divisions for each rotational component of a node:

$$a = \begin{cases} \left\lceil \frac{\delta'(r_{c_l})}{\delta'(r_{c_l-1})} \right\rceil & (\delta'(r_{c_l-1}) < (W_{\max} - W_{\min})) \\ 1 & (\text{else}). \end{cases} \quad (15)$$

Note that by introducing angular branching, the upper bound estimate in Eq. 12 becomes not strictly exact. However, as shown in the experiments, we empirically confirmed that it underestimates the upper bound in only 0.001 % of all cases.

Algorithm 2 Batched 3D-BBS

```

1:  $best\_score \leftarrow score\_threshold$ 
2: while  $C$  is not empty do
3:   Pop  $c$  from the queue  $C$ 
4:   if  $score(c) < best\_score$  then
5:     continue
6:   if  $c$  is a leaf node then
7:      $match \leftarrow c$ 
8:      $best\_score \leftarrow score(c)$ 
9:   else
10:    Branch: Split  $c$  into nodes  $C_{c_l}$ 
11:    Add  $C_{c_l}$  to  $C_{CPU}$ 
12:    if  $|C_{CPU}| > b$  then
13:       $C_{GPU} \leftarrow C_{CPU}$ 
14:      Compute and memorize a score for each element in  $C_{GPU}$ 
15:       $C_{CPU} \leftarrow C_{GPU}$ 
16:      Push  $C_{CPU}$  onto the queue  $C$ , sorted by score
17:      Clear the all elements of  $C_{CPU}$ .
18: return  $best\_score$  and  $match$  when set.

```

Search strategy: Although we first remove nodes with estimated upper bounds less than $score_threshold$ from the initial set, a large number of nodes remain in the queue. This causes a delay in finding the best solution for DFS and increases the processing time. Therefore, we employ BFS and sort C in descending order of score. Because BFS selects a node with the best upper bound score, it tends to find a solution quicker than DFS when the global best score is close to the maximum score (i.e., when the LiDAR scan has a large overlap with the map). However, in the middle levels, BFS requires branching a large number of high-score nodes because most of the input points overlap with inflated voxels. To speed up the branching process, we implement GPU-parallelized batch processing for the score calculation.

E. GPU-accelerated Batched Branch-and-Bound

We process the score calculations for multiple nodes in parallel on a GPU. To reduce the overhead of CPU-GPU data copy and synchronization, we implement batch processing. The procedure of our batched 3D-BBS is shown in Algorithm 2. The branched nodes are stored in the node set C_{CPU} on the CPU memory. When the size of C_{CPU} exceeds the batch size b (e.g., 10,000 nodes), C_{CPU} is transferred to the node set C_{GPU} on the GPU memory as a single memory block, and the scores of all nodes are calculated on the GPU. Subsequently, all the computed scores are sent back to C_{CPU} . Note that voxel maps H are copied to the GPU memory in advance to avoid run-time overheads.

IV. EXPERIMENTS

We evaluated the localization accuracy and processing time of the proposed method in simulated and real environments. To demonstrate the effectiveness of our algorithm, we used all combinations of processing types (CPU single-thread, CPU multi-thread (4 threads), and GPU), branching components (translation-only, translation + rotation), and search strategies (DFS and BFS) that resulted in nine configurations, as shown in Table I. The combination of CPU single-thread, translation-only branching, and DFS (configuration (a)) corresponds to a naive 3D extension of [4], and configuration (i) is our proposal. In the real environment, we

TABLE I: BnB algorithm configurations

Configuration	Processing type	Branching components	Search strategy
(a) (ext. [4])	CPU single-thread	Trans.	DFS
(b)	CPU multi-thread	Trans.	DFS
(c)	GPU	Trans.	DFS
(d)	CPU single-thread	Trans.	BFS
(e)	CPU multi-thread	Trans.	BFS
(f)	GPU	Trans.	BFS
(g)	CPU single-thread	Trans. and rot.	BFS
(h)	CPU multi-thread	Trans. and rot.	BFS
(i) (ours)	GPU	Trans. and rot.	BFS

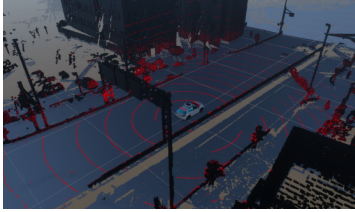


Fig. 3: Simulated environment.

compared the proposed configuration with the state-of-the-art global registration methods TEASER++ [1], Quatro [2], and FGR [3].

To evaluate the estimation result, we used two criteria: 1) the translation error was smaller than 2.0 m, and 2) the rotation error was smaller than 0.05 rad. If a result satisfies both the translation and rotation criteria, we consider it sufficiently accurate to be used as an initial estimate for local fine registration [22] to obtain a fine estimation result.

We used the same parameters for 3D-BBS through all the experiments. Considering the sizes of the experimental maps, we set $r = 1.0$ m and $l_{\max} = 6$. Note that r and $l_{\max}$ should be changed according to the required accuracy and size of the map, respectively. To reduce the computational cost of the score calculation, all LiDAR scans were downsampled to approximately 1,000 points using a voxel grid filter. In the GPU implementation, batch size b was empirically set to 10,000. The score threshold was set to 95 % of the total number of input points. The roll and pitch angle search range was respectively set to $(W_{\min}^{\alpha, \beta}, W_{\max}^{\alpha, \beta}) = (-0.02, 0.02)$ rad to compensate for the gravity direction estimation errors.

We implemented the proposed algorithm in C++ and CUDA. All processes were performed with Intel Core i7-10700K 3.8Ghz 32GB and NVIDIA GeForce RTX2060.

A. Simulated Environment

Experimental data: To demonstrate the performance of the proposed algorithm, we created LiDAR data and ground truth sensor poses in a simulated environment, as shown in Fig. 3. The 3D point cloud map data was captured in an urban environment containing many buildings, as shown in Fig. 4. The map size was $211.0 \times 729.7 \times 68.5$ m³. The total number of evaluation points was 295. We generated LiDAR scan point clouds with sensor parameters of the Velodyne VLP32C. We added random noise in the range of $[-0.01, 0.01]$ rad to the roll and pitch components of the

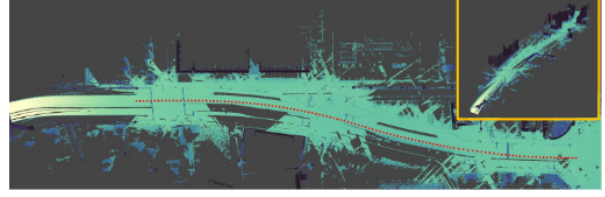


Fig. 4: 3D point cloud map in the experiment with the simulated environment. The map size is $211.0 \times 729.7 \times 68.5$ m³. The total number of evaluation points represented by red dots is 295.

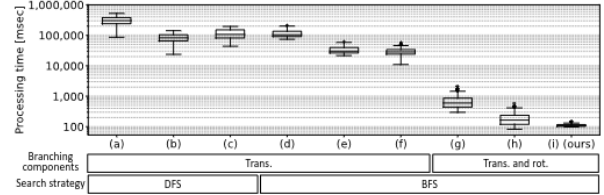


Fig. 5: Processing time of the configurations (a) - (i) in the simulated environment.

LiDAR scan point clouds to imitate the gravity direction estimation errors.

Results: Configurations (a) - (i) successfully estimated the sensor poses with translation and rotation errors smaller than the criterion threshold values for all evaluation points. Along with the accuracy evaluation, we evaluated how often the proposed rotational branching showed incorrect upper bound estimates and how much the incorrect estimates exceeded the true upper bound. We confirmed that the rotational branching showed incorrect upper bound estimates for only 0.001% of the cases, and the average error with respect to the true upper bound was about 1.5%. These results show that rotational branching introduces only a small approximation.

Fig. 5 shows the processing times of all configurations. Configurations (d) - (f) with BFS were slightly faster than configurations (a) - (c) with DFS. Because BFS first branches the node with the highest provisional score, it can find the global solution more quickly when the scan points have a large overlap with the map (i.e., the score of the solution is close to the maximum score). However, it was necessary to calculate a large number of nodes across all levels. In configurations (g) - (i), the combination of the rotational branch and BFS contributed to a significant reduction in processing time. Configuration (i) with batch processing reduced the processing time by more than 50 % from the configuration (g) with CPU single thread. Configurations (g) and (h) with CPU processing also demonstrated fast processing within less than 1 sec because a high best score in the simulated environment allows for faster pruning.

B. Real Environment

Experimental data: We then compared the proposed method with several other state-of-the-art methods in a real environment. Fig. 6 shows the outdoor point cloud map used in this experiment, containing buildings, parked vehicles, and

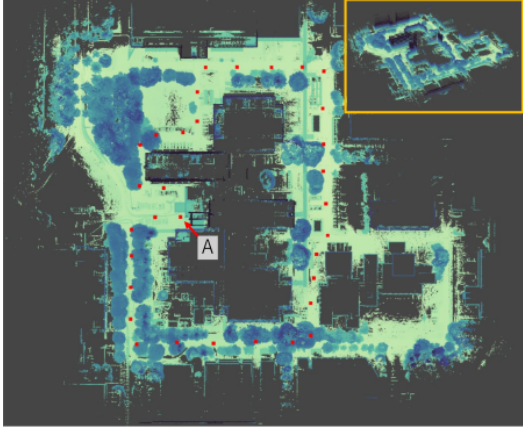


Fig. 6: 3D point cloud map in the experiment with real environment. The map size is $429.6 \times 404.3 \times 69.6 \text{ m}^3$. The total number of evaluation points represented by red dots is 32.

TABLE II: Success rate and processing time

Method	Trans.	Trans. and rot.	Processing time [msec]
TEASER++ [1]	16/32	11/32	$45,573 \pm 14,462$
Quatro [2]	18/32	18/32	$42,047 \pm 14,038$
FGR [3]	17/32	14/32	$43,677 \pm 15,283$
Conf. (i) (ours)	32/32	32/32	$878 \pm 1,128$

street trees. The map size was $429.6 \times 404.3 \times 69.6 \text{ m}^3$. The total number of evaluation points was 32. We used the Livox-Mid360 with a built-in IMU. We measured sensor data on a different day from when the map data was acquired, and thus the scan data had differences in parked vehicle positions and vegetation from the map data. The LiDAR scan was aligned in the direction of the average acceleration direction measured using the IMU. To obtain the ground truth sensor poses, we manually aligned each LiDAR scan with the map point cloud using local registration [22].

Estimation accuracy: Table II summarizes the success counts and processing times of the evaluated global localization methods. The processing time was calculated only for successful results. Global registration methods [1], [2], [3] succeeded at most 18 locations out of 32. Because there were many similar buildings in the map point cloud, these methods struggled with similar point features, which resulted in too many incorrect correspondences, and showed incorrect estimation results far from the ground truth locations several times. In contrast, the proposed 3D-BBS successfully estimated the correct sensor locations for all evaluation data even under environmental changes, e.g., vegetation and dynamic obstacles. Moreover, configurations (a) - (h) were also successful at all evaluation points in the real environment, similar to the simulation environment.

Processing time: Fig. 7 shows the processing times of all configurations. We observe that the proposed configuration (i) performed the fastest processing in the real outdoor environment. Moreover, there was a large difference between configurations (g) and (h) with the CPU and configuration

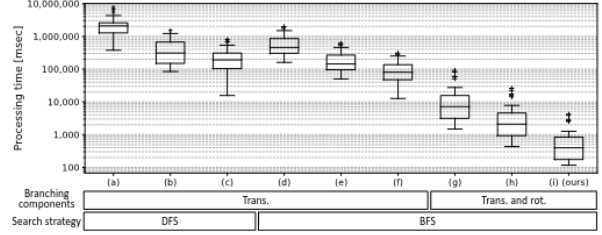


Fig. 7: Processing time of the configurations (a) - (i) in the real environment.

TABLE III: Processing breakdown [msec]

Process	Procedure	Average
Preprocessing	Create voxel maps	$9,272.2 \pm 49.8$
	Set source point cloud	30.5 ± 0.7
	Total	$9,302.7 \pm 49.7$
Localization	Initial nodes calculation	2.7 ± 0.1
	Find best score	119.8 ± 1.4
	Pop remaining queue	50.9 ± 2.3
	Total	173.4 ± 3.1

(i) with the GPU. This was because as the number of nodes to be branched increased, the overhead of the GPU batch processing got canceled, resulting in a large performance gap between the CPU and GPU implementations. However, unlike the simulated environment, the interquartile range was large overall. The more outliers included in the LiDAR scan, the more it affects the processing time. To further improve 3D-BBS performance, it is necessary to consider the inconsistency between the map and LiDAR scans.

Table III shows the breakdown of the processing time of 3D-BBS with the proposed configuration (i) at evaluation point A, as shown in Fig. 6. The preprocessing, which needed to be performed for the map only once, was completed within 10 sec, including the memory copy of the multi-resolution voxel map from the CPU to the GPU. The set source point includes the construction of a graph structure for an efficient parallel score calculation. In the localization step, the number of initial nodes was significantly reduced thanks to the rotational branching, and the initial node calculation took only 2.7 msec. Moreover, the branching operation accelerated by the batch process allowed us to find the best score in 119.8 msec. The localization process was completed by popping the remaining queue in 50.9 msec.

V. CONCLUSION

This paper proposed a 3D global localization approach based on BnB algorithm. We used a sparse hash table to overcome the memory increase in hierarchical 3D voxel maps. Moreover, we proposed roto-translational branching and batched processing to reduce the processing time. The experimental results showed that 3D-BBS accurately estimated the global pose and completed the localization process in a second. In future work, we plan to extend the proposed method to handle extreme cases (e.g., degenerated areas or LiDAR scan occlusions).

REFERENCES

- [1] H. Yang, J. Shi, and L. Carlone, "TEASER: Fast and certifiable point cloud registration," *IEEE Transactions on Robotics*, vol. 37, no. 2, pp. 314–333, apr 2021.
- [2] H. Lim, S. Yeon, S. Ryu, Y. Lee, Y. Kim, J. Yun, E. Jung, D. Lee, and H. Myung, "A single correspondence is enough: Robust global registration to avoid degeneracy in urban environments," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, may 2022.
- [3] Q.-Y. Zhou, J. Park, and V. Koltun, "Fast global registration," in *European Conference on Computer Vision (ECCV)*. Springer International Publishing, 2016, pp. 766–782.
- [4] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2d LIDAR SLAM," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, may 2016.
- [5] H. Yin, X. Xu, S. Lu, X. Chen, R. Xiong, S. Shen, C. Stachniss, and Y. Wang, "A survey on global lidar localization: Challenges, advances and open problems," in *arXiv preprint arXiv:2302.07433*, 2023.
- [6] R. B. Rusu, N. Blodow, and M. Beetz, "Fast point feature histograms (FPFH) for 3d registration," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, may 2009.
- [7] S. Salti, F. Tombari, and L. D. Stefano, "SHOT: Unique signatures of histograms for surface and texture description," *Computer Vision and Image Understanding*, vol. 125, pp. 251–264, aug 2014.
- [8] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," in *Readings in Computer Vision*. Elsevier, 1987, pp. 726–740.
- [9] X. Chen, I. Vizzo, T. Labe, J. Behley, and C. Stachniss, "Range image-based LiDAR localization for autonomous vehicles," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, may 2021.
- [10] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, "Monte carlo localization for mobile robots," in *IEEE International Conference on Robotics and Automation (ICRA)*, vol. 2. IEEE, 1999, pp. 1322–1328.
- [11] G. Kim and A. Kim, "Scan context: Egocentric spatial descriptor for place recognition within 3d point cloud map," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, oct 2018.
- [12] X. Chen, T. Läbe, A. Milioto, T. Röhling, J. Behley, and C. Stachniss, "OverlapNet: a siamese network for computing LiDAR scan similarity with applications to loop closing and localization," *Autonomous Robots*, vol. 46, no. 1, pp. 61–81, aug 2021.
- [13] J. Ma, J. Zhang, J. Xu, R. Ai, W. Gu, and X. Chen, "Overlaptransformer: An efficient and yaw-angle-invariant transformer network for lidar-based place recognition," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6958–6965, 2022.
- [14] Y. Cui, X. Chen, Y. Zhang, J. Dong, Q. Wu, and F. Zhu, "BoW3d: Bag of words for real-time loop closing in 3d LiDAR SLAM," *IEEE Robotics and Automation Letters*, vol. 8, no. 5, pp. 2828–2835, may 2023.
- [15] Y. Cui, Y. Zhang, J. Dong, H. Sun, and F. Zhu, "Link3d: Linear keypoints representation for 3d lidar point cloud," in *arXiv preprint arXiv:2206.05927*, 2022.
- [16] A. H. Land and A. G. Doig, "An automatic method of solving discrete programming problems," *Econometrica*, vol. 28, no. 3, p. 497, jul 1960.
- [17] D. R. Morrison, S. H. Jacobson, J. J. Sauppe, and E. C. Sewell, "Branch-and-bound algorithms: A survey of recent advances in searching, branching, and pruning," *Discrete Optimization*, vol. 19, pp. 79–102, feb 2016.
- [18] C. Olsson, F. Kahl, and M. Oskarsson, "Branch-and-bound methods for euclidean registration problems," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 31, no. 5, pp. 783–794, may 2009.
- [19] J. Yang, H. Li, and Y. Jia, "Go-ICP: Solving 3d registration efficiently and globally optimally," in *IEEE International Conference on Computer Vision (ICCV)*. IEEE, dec 2013, pp. 1457–1464.
- [20] P. Besl and N. D. McKay, "A method for registration of 3-d shapes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, feb 1992.
- [21] M. Teschner, B. Heidelberger, M. Müller, D. Pomeranets, and M. Gross, "Optimized spatial hashing for collision detection of deformable objects," in *Vision, Modeling, and Visualization (VMV)*, 2003, pp. 47–54.
- [22] A. Segal, D. Haehnel, and S. Thrun, "Generalized-ICP," in *Robotics: Science and Systems*. Robotics: Science and Systems Foundation, jun 2009.